

Course Folder Management System

Complete Windows 10 Deployment Guide - fresh machine, end to end, copy & paste (PowerShell)

University Institute of Information Technology (UIIT), PMAS Arid Agriculture University, Rawalpindi

Prepared by: Noman Amjad

This is the single, complete guide for putting the Course Folder Management System onto a **brand-new Windows 10 computer** that will act as the server on the university's local network. It is written for a **non-technical team**: every step says what it does, gives the exact command to copy and paste, and lists an alternative in case the first way does not work. Do the sections **in order, top to bottom**. Do not skip any.

Very important - internet is needed only while setting up. Installing the tools and building the app downloads about 500 MB from the internet. So the server **must be connected to the internet during setup**. **After** the first successful build, the app runs completely on the local network and never needs the internet again. So: connect to the internet, finish this guide, then the machine can be moved to the offline LAN.

Before you begin, make sure you have: a Windows 10 (64-bit) PC you can log in to as an **Administrator**; 8 GB RAM or more (this machine has 32 GB); an internet connection for now; and about 45-60 minutes. You do **not** need any programming knowledge - just follow along.

Step 1 - Open PowerShell as Administrator

What this does: opens the command window with the permissions needed to install software.

1. Click **Start** and type `powershell`.
2. Right-click **Windows PowerShell** -> **Run as administrator** -> **Yes**.
3. The title bar must read *Administrator: Windows PowerShell*. Keep this window open for the whole guide.

Check the machine is 64-bit (must print `True`):

```
[System.Environment]::Is64BitOperatingSystem
```

Step 2 - Turn on virtualization

What this does: lets Docker run its small Linux engine. Usually already on.

1. Press `Ctrl+Shift+Esc` -> **Performance** tab -> **CPU**.
2. Find **Virtualization**. If it says **Enabled**, go to Step 3.
3. If **Disabled**: restart, open BIOS/UEFI (tap `Del`, `F2`, or `F10` at power-on), turn on **Virtualization Technology** (Intel VT-x / AMD SVM), save, and reboot.

Step 3 - Install WSL 2 (Docker's engine)

What this does: installs the Linux layer Docker needs.

```
wsl --install
```

When it finishes, **restart the computer**. After the restart, open Administrator PowerShell again and run:

```
wsl --set-default-version 2
```

Alternative if `wsl --install` says "not recognized": turn the features on manually, then reboot and retry:

```
dism.exe /online /enable-feature /featurename:Microsoft-Windows-Subsystem-Linux /all /norestart  
dism.exe /online /enable-feature /featurename:VirtualMachinePlatform /all /norestart
```

Step 4 - Install Docker Desktop

What this does: installs the program that builds and runs the whole system. Download it using the **FIRST** option that works.

4.1 Download the installer

Option A - curl (built into Windows 10, most reliable):

```
curl.exe -L "https://desktop.docker.com/win/main/amd64/Docker%20Desktop%20Installer.exe" -o "$env:USERPROFILE\Downloads\DockerInstaller.exe"
```

Option B - PowerShell download:

```
$ProgressPreference = 'SilentlyContinue' Invoke-WebRequest -Uri "https://desktop.docker.com/win/main/amd64/Docker%20Desktop%20Installer.exe" -OutFile "$env:USERPROFILE\Downloads\DockerInstaller.exe"
```

Option C - BITS transfer (good for large files):

```
Start-BitsTransfer -Source "https://desktop.docker.com/win/main/amd64/Docker%20Desktop%20Installer.exe" -Destination "$env:USERPROFILE\Downloads\DockerInstaller.exe"
```

Option D - winget (if available; downloads AND installs, then skip 4.2):

```
winget install -e --id Docker.DockerDesktop
```

Option E - just use a browser: open <https://www.docker.com/products/docker-desktop/> and click **Download for Windows**. The file lands in your Downloads folder.

4.2 Run the installer

```
Start-Process "$env:USERPROFILE\Downloads\DockerInstaller.exe" -Wait
```

Keep **Use WSL 2** ticked, finish, and **restart if asked**.

Alternative - silent install (no clicking):

```
Start-Process "$env:USERPROFILE\Downloads\DockerInstaller.exe" -Wait -ArgumentList "install","--quiet","--accept-license"
```

4.3 Start Docker and make the server come back by itself

1. Open **Docker Desktop** from Start. Accept the terms.
2. Wait until it says **Engine running** (the whale icon in the tray is steady).
3. Open **Settings (gear)** -> **General** -> tick **Start Docker Desktop when you sign in**.

With that ticked, and because the app already uses Docker's **restart: unless-stopped** setting, the server starts again on its own whenever Windows is signed in - after a normal restart or a power cut. No command is needed.

Make it fully automatic (no password typed on power-on). Docker Desktop only starts *after someone signs in to Windows*. So the machine can sign in by itself, turn on Windows automatic sign-in for the server account:

1. Use a **local** Windows account for the server (not a Microsoft account) if possible.
2. Press **Win + R**, type **netplwiz**, press **Enter**.
3. Select the account, **untick** "Users must enter a user name and password to use this computer", click **Apply**.
4. Type the account password twice, click **OK**.

If that checkbox is missing: open **Settings** -> **Accounts** -> **Sign-in options**, turn **off** "Require Windows Hello sign-in for Microsoft accounts", restart the computer, then try **netplwiz** again.

After a power cut or a restart: just switch the machine on. Windows signs in by itself, Docker Desktop starts, and the server comes back - **wait about 1-2 minutes** for Docker to finish starting, then `http://YOUR-SERVER-IP:8000` works again. If it is ever still down, open PowerShell and run:

```
cd C:\course-folder docker compose -f docker-compose.prod.yml up -d
```

Security note: automatic sign-in means anyone who powers this machine on gets a signed-in Windows session - keep the server in a locked room.

4.4 Check Docker works

```
docker --version docker compose version docker run hello-world
```

The last command should print *"Hello from Docker!"*. If it does, Docker is ready.

Step 5 - Download the application code

What this does: puts the project files on the server. Use ONE option.

Option A - download the ZIP (simplest, no extra tools):

```
curl.exe -L "https://github.com/780Noman/course-folder-management-system/archive/refs/heads/main.zip" -o
"$env:USERPROFILE\Downloads\app.zip" Expand-Archive "$env:USERPROFILE\Downloads\app.zip" -DestinationPath "C:\"
-Force Rename-Item "C:\course-folder-management-system-main" "C:\course-folder"
```

Option B - with Git (the repository is public, so no login is needed):

```
winget install -e --id Git.Git cd C:\ git clone https://github.com/780Noman/course-folder-management-system.git
course-folder
```

Option C - from a USB drive: copy the project folder to `C:\course-folder` (you do not need the `.venv` folder or any video files).

Now move into the folder. **Every command after this runs from here:**

```
cd C:\course-folder
```

Step 6 - Create the settings file (.env)

What this does: writes the server's settings (a secret key, a database password, and this machine's network address). The command below makes them for you automatically - you only type the IP address.

6.1 Find this machine's IP address

```
ipconfig
```

Under your active network adapter, read the **IPv4 Address**, for example `192.168.1.50`. Keep it handy for the next command.

6.2 Create the .env file automatically

Copy and paste this whole block. It will ask for the IP address, generate a strong secret key and database password, and write the file:

```
$ip = Read-Host "Enter this server's IPv4 address (example 192.168.1.50)" $secret = -join
((48..57)+(65..90)+(97..122) | Get-Random -Count 50 | ForEach-Object {[char]$_}) $dbpass = -join
((48..57)+(65..90)+(97..122) | Get-Random -Count 24 | ForEach-Object {[char]$_}) @"
DJANGO_SETTINGS_MODULE=config.settings.prod SECRET_KEY=$secret DEBUG=False ALLOWED_HOSTS=$ip
DATABASE_URL=postgres://postgres:$dbpass@db:5432/coursefolders POSTGRES_DB=coursefolders POSTGRES_USER=postgres
POSTGRES_PASSWORD=$dbpass AWS_ACCESS_KEY_ID= AWS_SECRET_ACCESS_KEY= AWS_STORAGE_BUCKET_NAME= AWS_S3_ENDPOINT_URL=
AWS_S3_REGION_NAME=auto EMAIL_HOST= EMAIL_PORT=587 EMAIL_HOST_USER= EMAIL_HOST_PASSWORD= EMAIL_USE_TLS=True
DEFAULT_FROM_EMAIL=Course Folder System <no-reply@uiit.edu.pk> MAX_UPLOAD_MB=100 SIGNED_URL_TTL=300 PDF_ENGINE=auto
ENABLE_HTTPS=False CSRF_TRUSTED_ORIGINS=http://$ip:8000 WEB_CONCURRENCY=3 GUNICORN_BIND=0.0.0.0:8000
GUNICORN_TIMEOUT=60 "@ | Set-Content -Encoding ascii .env Write-Host "Created .env for $ip" -ForegroundColor Green
```

You should see a green *"Created .env for ..."* line. Check it if you like:

```
Get-Content .env
```

Alternative - do it by hand: copy the sample and edit it in Notepad, setting SECRET_KEY, POSTGRES_PASSWORD (put the same password inside DATABASE_URL too), ALLOWED_HOSTS, and CSRF_TRUSTED_ORIGINS:

```
Copy-Item .env.example .env notepad .env
```

Leave ENABLE_HTTPS=False as it is - the site is served over plain `http://` on the LAN. The email lines can stay empty (see the last section). **Never share the .env file - it holds the passwords.**

Step 7 - Give the machine a fixed IP address

What this does: keeps the server's address from changing, so people can always reach it at the same web address.

- **Best - ask IT for a "DHCP reservation"** for this machine (they need its Physical Address, shown by `ipconfig /all`).
- **Or set it in Windows:** Settings -> **Network & Internet** -> **Change adapter options** -> right-click your adapter -> **Properties** -> **Internet Protocol Version 4 (TCP/IPv4)** -> **Use the following IP address**, and enter the IP, subnet mask (`255.255.255.0`), gateway, and DNS from your IT section.

Step 8 - Build and start the system

What this does: builds the app, creates the database, and starts everything. This is the step that needs the internet (about 500 MB, one time).

```
docker compose -f docker-compose.prod.yml up -d --build
```

Wait a few minutes for it to finish. Then confirm both parts are running:

```
docker compose -f docker-compose.prod.yml ps
```

You should see `db` and `web`, both *running / healthy*. To watch the start-up log:

```
docker compose -f docker-compose.prod.yml logs -f web
```

When the log stops scrolling and shows it is listening on port 8000, press `Ctrl+C` to stop watching (this does NOT stop the server).

Step 9 - Create the first administrator (focal person)

What this does: makes the main account that can log in and manage everyone. This is the only account made by command.

```
docker compose -f docker-compose.prod.yml exec web python manage.py createsuperuser
```

Type an **email** (this is the login name), a **name**, and a **password** (typed twice - it will not show on screen; that is normal).

Step 10 - Open the firewall for port 8000

What this does: lets other computers on the network reach the site.

```
New-NetFirewallRule -DisplayName "Course Folder 8000" -Direction Inbound -Protocol TCP -LocalPort 8000 -Action Allow
```

Step 11 - Test that it works

1. **On the server**, open a browser to `http://localhost:8000` - the login page should appear.
2. **From another PC or phone on the same network**, open `http://YOUR-IP:8000` (for example `http://192.168.1.50:8000`). The same page should load.
3. Log in with the administrator email and password from Step 9.
4. Create a test faculty (next section) and log in as them on another device to confirm the full flow.

Step 12 - Adding and managing users (no email needed)

Because the server is offline, the administrator sets passwords directly - the system does not send emails.

- **Add a user:** *Faculty* -> *Add faculty* -> type their name, email, role, and an **initial password** -> **Create account**. Tell the person their email and password.
- **The user changes their own password** from the change-password page after signing in.
- **Someone forgot their password?** *Faculty list* -> *Reset password* next to their name -> set a new one and share it.

Step 13 - Everyday running (start, stop, logs)

Task	Command (run from C:\course-folder)
See what is running	<code>docker compose -f docker-compose.prod.yml ps</code>
View the log	<code>docker compose -f docker-compose.prod.yml logs -f web</code>
Stop (data is kept)	<code>docker compose -f docker-compose.prod.yml down</code>
Start again	<code>docker compose -f docker-compose.prod.yml up -d</code>
Restart just the app	<code>docker compose -f docker-compose.prod.yml restart web</code>

The database and all uploaded documents are stored safely in Docker volumes and survive stop/start, reboots, and updates. With Docker set to start at sign-in (Step 4.3) the whole system comes back on its own after a reboot.

For a server nobody logs into daily, ask IT to enable automatic Windows sign-in for a dedicated local account, so Docker Desktop starts after a reboot without anyone signing in.

Step 14 - Backups (do this regularly)

Save the database to a file (make a folder such as D:\backups first):

```
docker compose -f docker-compose.prod.yml exec -T db pg_dump -U postgres coursefolders >
D:\backups\coursefolders.sql
```

Back up the uploaded documents (stored in the `media` volume):

```
docker run --rm -v course-folder_media:/data -v D:\backups:/backup busybox tar czf /backup/media.tar.gz -C /data .
```

(Check the exact volume name with `docker volume ls`; it is the folder name plus `_media`.) A fuller routine is in `docs/BACKUPS.md`.

Step 15 - Updating to a newer version later

*What this does: brings the newest code onto the server and restarts it. Your data - all accounts, passwords, uploaded documents, and issued certificates - lives in Docker volumes and is **not touched**. Migrations and static files are applied automatically on restart, so there is nothing else to run.*

15.1 Before you start

1. Make sure **Docker Desktop** is running (whale icon steady, *Engine running*).
2. Open **PowerShell as Administrator** and move into the app folder:

```
cd C:\course-folder
```

15.2 Get the latest code (use the SAME method you installed with)

Option A - if you installed with Git (simplest and safest):

```
git pull
```

This updates the code only; your `.env` settings file is left alone.

Option B - if you installed from the ZIP (no Git):

```
# 1) Keep a safety copy of your settings file Copy-Item .\.env "$env:USERPROFILE\Downloads\env-backup.txt" -Force #
2) Download and unpack the latest code curl.exe -L
"https://github.com/780Noman/course-folder-management-system/archive/refs/heads/main.zip" -o
"$env:USERPROFILE\Downloads\app.zip" Expand-Archive "$env:USERPROFILE\Downloads\app.zip" -DestinationPath
"$env:USERPROFILE\Downloads\app-new" -Force # 3) Copy the new files over your folder (your .env is NOT in the zip,
so it stays) Copy-Item "$env:USERPROFILE\Downloads\app-new\course-folder-management-system-main\*"
"C:\course-folder\" -Recurse -Force
```

15.3 (Optional) Raise the upload limit to 100 MB

Only if your `.env` still says `MAX_UPLOAD_MB=50` and you want the larger limit (for PowerPoint / scanned files). Open it, change the line to `MAX_UPLOAD_MB=100`, save, and close:

```
notepad C:\course-folder\.env
```

15.4 Rebuild and restart

```
docker compose -f docker-compose.prod.yml up -d --build
```

This takes about 1-3 minutes. On start-up the system applies any database migrations and refreshes its files automatically - no other command is needed.

15.5 Check it worked

```
docker compose -f docker-compose.prod.yml ps docker compose -f docker-compose.prod.yml logs --tail=30 web
```

- `ps`: both `db` and `web` should show **Up**.
- `Logs`: you should see *"Running database migrations... Collecting static files..."*.
- Open `http://YOUR-SERVER-IP:8000` in a browser, sign in, and confirm the new features work (e.g. the **"Bulk upload from a folder"** panel on a course folder).

If something looks wrong (rollback). Your data is safe in the volumes. To go back to the previous code: with Git run `git checkout` of the earlier version and rebuild (Step 15.4); with the ZIP, unpack the previous version and rebuild. Nothing is lost.

Do not delete the `C:\course-folder` folder or the `.env` file, and do not run `docker compose down -v` - the `-v` flag would erase the database and uploaded files. A plain `up -d --build` never touches your data.

Step 16 - If something goes wrong

Problem	What to do
<code>docker</code> is not recognized	Docker Desktop is not running. Open it, wait for <i>Engine running</i> , retry.
Build fails while downloading	No internet. Connect the machine to the internet for this step; it can go offline afterwards.
<code>failed to resolve host 'db'</code>	You ran the app outside Docker. Always use the <code>docker compose ...</code> commands in this guide.
Other PCs cannot open the site	Check the firewall rule (Step 10), that you used the correct IP, and that both devices are on the same network.
"Bad Request (400)"	The IP is not in <code>.env</code> . Re-do Step 6 with the correct IP, then <code>docker compose -f docker-compose.prod.yml up -d --force-recreate web</code> .
"Too many failed sign-in attempts"	A safety lock after several wrong passwords. Wait about 15 minutes, then use the correct password.
Port 8000 already in use	Another program uses it. Close that program, or ask for the port to be changed in <code>docker-compose.prod.yml</code> .

Email is optional. This offline setup does not need email - the administrator sets passwords directly (Step 12). If the server is ever connected to the internet and you want automatic emails, fill the `EMAIL_*` lines in `.env` and run `docker compose -f docker-compose.prod.yml up -d --force-recreate web`. See the separate *Email (SMTP) Configuration Guide* for the details.

That's the whole process. After Step 11 the system is live on the network; Steps 12-16 are for daily use and upkeep. Keep this guide with the server.